B.TECH 3RD-YEAR AI/ML PROJECT

AI-Based Indian Sign Language Alphabet Recognition and Voice Conversion System

PROJECT DESIGN REPORT (PDR)

Project objectives, scope, requirements, feasibility and development plan

Project scope: Recognition of ISL alphabet/fingerspelling classes A–Z from images or webcam frames, conversion of recognized letters into editable text, and English text-to-speech output.

Document status: Planning / proposed design. This document describes the intended system. It does not claim that implementation, model training, testing or deployment has already been completed.

Prepared for group project planning and development. Team members, institute details, guide name and submission date can be added by the group.

Document Control

Item	Details
Project title	AI-Based Indian Sign Language Alphabet Recognition and Voice Conversion System
Document	Project Design Report (PDR)
Version	1.0 – Initial planning draft
Status	Proposed; to be updated after team approval
Prepared for	B.Tech 3rd-year AI/ML academic project
Target classes	26 alphabet labels: A–Z

1. Executive Summary

The project proposes an educational web application that recognizes selected Indian Sign Language (ISL) alphabet/fingerspelling hand poses using a Convolutional Neural Network (CNN). A user will provide a hand-sign image or use a browser webcam. The backend will preprocess the input and run the trained model to return a predicted letter and confidence score. Recognized letters can be collected into an editable text area. When the user presses Speak, the browser's SpeechSynthesis API will read the text aloud in English.

The planned solution combines machine learning, computer vision, a Python API backend and a React-based frontend. The project will be developed incrementally, evaluated using real test data and documented with its limitations.

2. Problem Statement

People who communicate through sign language may face communication barriers when interacting with people who do not understand their signs. This project explores a limited technical prototype that can recognize ISL alphabet hand poses and convert a sequence of recognized letters into text and speech. The project is not intended to replace interpreters or translate complete natural ISL conversations.

3. Aim and Objectives

- Develop an image-classification model for the planned ISL alphabet classes A–Z.
- Collect and inspect a public dataset and a small consent-based custom dataset.
- Preprocess images consistently for CNN training and inference.
- Train and evaluate a CNN using separate training, validation and testing data.
- Expose model inference through a FastAPI backend.
- Build a React web interface for image upload and webcam-based recognition.
- Provide a gesture-to-text builder with editing controls.
- Convert the user-approved text into English speech using browser speech synthesis.
- Report actual performance, test conditions, errors and project limitations.

4. Project Scope

4.1 Included in the initial scope

- Alphabet/fingerspelling recognition for target labels A–Z, subject to dataset and sign-convention audit.
- Static image upload and prediction.
- Browser webcam frame capture and backend prediction.
- Predicted label and model confidence display.
- Text builder with Add, Space, Delete, Clear and Speak controls.
- English text-to-speech output.

- Model evaluation outputs such as accuracy, precision, recall, F1-score and confusion matrix.
- Basic prediction history and model information, if time permits.

4.2 Excluded from the initial scope

- Full natural ISL sentence translation.
- Automatic understanding of ISL grammar, facial expression, context or complete body language.
- Guaranteed real-time interpretation for all users and environments.
- Replacement of qualified sign-language interpreters.
- A claim that every alphabet sign can be identified reliably from one still image.

Some letters may involve movement or have dataset-specific representations. The team must inspect the selected dataset and validate the sign conventions before finalizing the still-image baseline. If motion is required, it will be recorded as a limitation or considered as a separately approved extension.

5. Stakeholders and Intended Users

Stakeholder	Interest / role
Project team	Design, implement, test and explain the system.
Project guide / evaluator	Review technical approach, implementation, results and documentation.
Volunteers / data contributors	May contribute hand-sign images after informed consent.
Demo users	Try image upload, webcam recognition, text editing and speech output.
Potential accessibility users	May provide usability feedback; the prototype must not be presented as a validated assistive product.

6. Functional Requirements

ID	Requirement	Priority
FR-01	The system shall accept supported image files (PNG/JPG/JPEG) for prediction.	High
FR-02	The system shall preprocess input images consistently with the trained model.	High
FR-03	The system shall return a predicted alphabet label and confidence score.	High
FR-04	The frontend shall request webcam permission and capture frames in the browser.	High
FR-05	The frontend shall send captured frames to the backend for prediction.	High
FR-06	The user shall be able to add recognized letters to an editable text builder.	High
FR-07	The user shall be able to insert spaces, delete characters and clear text.	High
FR-08	The system shall speak the text in English after the user activates Speak.	High
FR-09	The application should display real evaluation metrics when available.	Medium
FR-10	The application may store prediction history using SQLite.	Medium

7. Non-Functional Requirements

- Usability: interface controls and status messages should be clear to a first-time user.
- Maintainability: code should be divided into understandable frontend, backend and training modules.
- Reliability: invalid files, unavailable model and API errors should be handled clearly.
- Performance: prediction response should be measured on the actual development machine; no unverified speed target is promised.
- Privacy: custom image collection must use consent; avoid collecting unnecessary personal information.
- Portability: setup instructions should allow another team member to run the project.

• Honesty: no fabricated predictions, training metrics, dataset counts or completion claims.

8. Proposed Technology Stack

Area	Technology	Purpose
ML	Python, TensorFlow/Keras	CNN model training and inference.
Image processing	OpenCV, NumPy	Image loading, resizing, colour handling and array processing.
Evaluation	scikit-learn, Matplotlib	Classification metrics, confusion matrix and plots.
Backend	FastAPI, Uvicorn	REST APIs and model inference service.
Frontend	React, Vite, JavaScript, Tailwind CSS	Web pages, upload, webcam and text builder.
Charts	Recharts or Chart.js	Display real evaluation results.
History	SQLite	Optional application prediction records.
Voice	Browser SpeechSynthesis API	English text-to-speech.

9. Dataset Plan

The planned primary public dataset is RealSign – Indian Sign Language Dataset: https://github.com/RealSign62/RealSign-Indian-Sign-Language-Dataset. Its repository description reports A–Z fingerspelling images and a train/validation/test folder structure. The published per-class counts and licence must be checked against the current repository and the downloaded archive. Actual file counts will be verified locally before training.

The team also plans to collect images from volunteers. A suggested initial collection target is approximately five volunteers and 30–40 images per class per volunteer. This is a planning estimate, not a completed collection. The group should obtain consent, document collection conditions and keep a separate holdout set for evaluating performance on unseen people where practical.

- Training data is used to learn model parameters.
- Validation data is used to monitor training and compare model settings.
- Testing data is reserved for final evaluation.
- Near-identical images from the same capture sequence should not be split across training and testing.
- Dataset source, licence, label mapping and split method must be documented.

10. Feasibility Assessment

Type	Assessment
Technical	Feasible as a student prototype using established Python, TensorFlow, FastAPI and React tools. Model quality depends on data quality, sign definitions and hardware.
Operational	The app can be demonstrated on a laptop with a webcam or uploaded images. Browser camera permissions and local API setup must be tested.
Economic	The planned stack is based mainly on free/open-source tools and built-in browser speech synthesis. Hardware and internet access may still be needed.
Schedule	A phase-wise approach reduces integration risk. The schedule must be adapted to team size, available time and actual dataset issues.

11. Risks and Mitigation

Risk	Possible impact	Mitigation
Incorrect or inconsistent labels	Model learns wrong hand-sign mapping.	Audit sample images and verify labels with reliable ISL references or knowledgeable reviewers.

Risk	Possible impact	Mitigation
Signer/data leakage	Test score appears better than real generalization.	Use signer-independent split where possible; reserve custom users for external testing.
Dynamic signs in a still-image dataset	Single-frame model may miss movement information.	Document the limitation or plan a separate sequence-based extension.
Lighting/background variation	Unstable predictions in live use.	Collect varied data and test under different conditions.
Webcam repeated predictions	One gesture adds the same letter many times.	Use stability checks, cooldown/debounce and explicit add controls.
Dependency/version issues	Installation or runtime failures.	Use a virtual environment, record versions and test setup on a clean machine.
Model not ready during UI work	Frontend may be tempted to show fake results.	Use a clearly labelled unavailable/demo state; never represent mock output as AI prediction.

12. Development Plan and Deliverables

Phase	Main activities	Expected deliverable
1. Setup	Install tools, create repository and folders.	Working development environment and project skeleton.
2. Dataset	Download, inspect, document and collect data.	Verified dataset and dataset notes.
3. Preprocessing	Resize, normalize, label and augment training data.	Reusable preprocessing pipeline.
4. CNN	Build, train and save baseline model.	Trained model artifact.
5. Evaluation	Test and generate metrics/plots.	Honest evaluation report.
6. Backend	Build model and prediction APIs.	Working FastAPI service.
7. Frontend	Build pages, upload and user controls.	Working React interface.
8. Integration	Connect camera, prediction, text builder and speech.	End-to-end prototype.
9. Submission	Test, document, prepare PPT and viva.	Project submission package.

13. Acceptance Criteria

- The team can install and run the project using documented instructions.
- A verified model can load and return a class prediction for a supported image.
- The API returns structured prediction information and handles invalid input.
- The frontend can upload an image and display the backend response.
- If webcam support is implemented, browser frames are sent to the backend and results are displayed.
- Text builder controls work and user-approved text can be spoken in English.
- Evaluation metrics are generated from a held-out test set and can be traced to saved results.
- Limitations and dataset provenance are included in the final report.

14. Assumptions and Open Decisions

- The initial class list is A–Z; exact hand configurations and any motion-dependent classes require dataset audit.
- The public dataset archive will be downloaded and inspected by the team.
- The number of volunteers and custom image collection conditions are not yet finalized.
- The final TensorFlow/Python versions will be selected based on compatibility at setup time.
- Final API route names, database schema and UI design may be refined during implementation.
- Team member names, institute details, guide details and final schedule are to be filled in.

15. References / Starting Resources

- RealSign ISL Dataset: https://github.com/RealSign62/RealSign-Indian-Sign-Language-Dataset
- Mendeley static ISL dataset: https://data.mendeley.com/datasets/7tsw22y96w/1
- ISLAN dataset: https://data.mendeley.com/datasets/rc349j45m5/1
- TensorFlow: https://www.tensorflow.org/
- FastAPI: https://fastapi.tiangolo.com/
- React: https://react.dev/
- OpenCV: https://opencv.org/

This PDR is an initial planning document. Update it when the team approves the architecture, confirms the dataset, assigns responsibilities and records actual implementation progress.